

ASSIGNMENT-10.5

2303a51215

20/02/26

Aravind

Batch-04

Task Description #1 –

Variable Naming Issues

Task: Use AI to improve unclear variable names.

Sample Input Code:

```
def f(a, b):
```

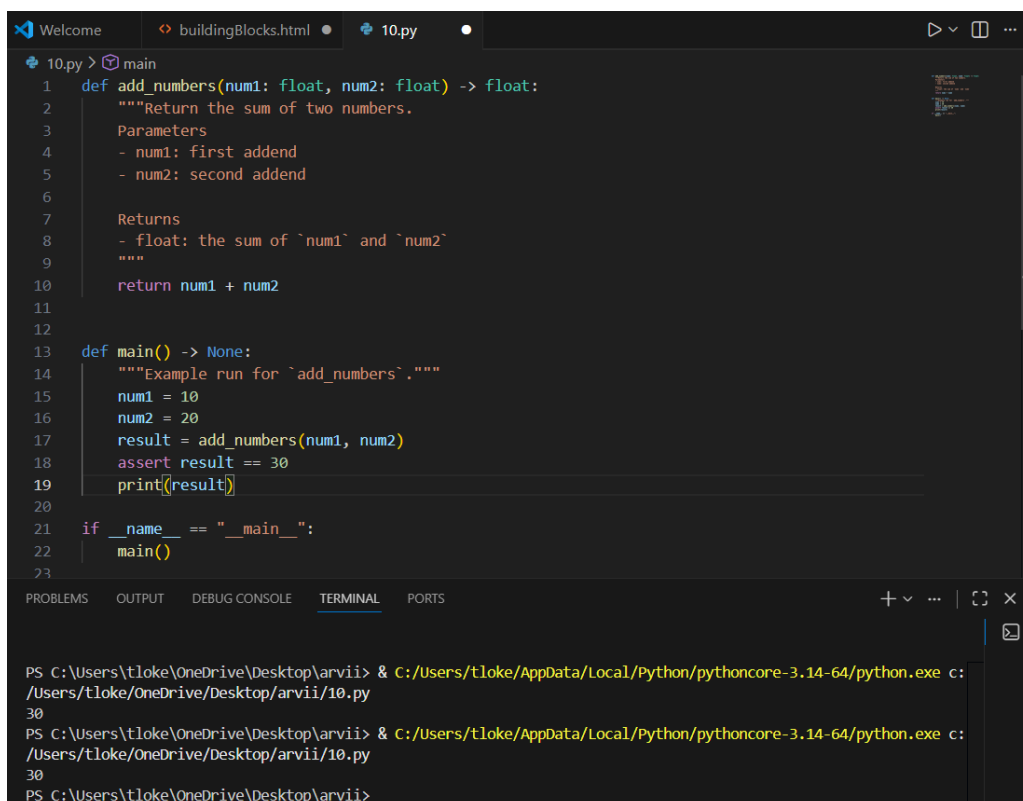
```
    return a + b
```

```
print(f(10, 20))
```

Expected Output:

- Code rewritten with meaningful function and variable names.

CODE:



```
10.py > main
1 def add_numbers(num1: float, num2: float) -> float:
2     """Return the sum of two numbers.
3     Parameters
4     - num1: first addend
5     - num2: second addend
6
7     Returns
8     - float: the sum of `num1` and `num2`
9     """
10    return num1 + num2
11
12
13 def main() -> None:
14     """Example run for `add_numbers`."""
15     num1 = 10
16     num2 = 20
17     result = add_numbers(num1, num2)
18     assert result == 30
19     print(result)
20
21 if __name__ == "__main__":
22     main()
23
```

```
PS C:\Users\tloke\OneDrive\Desktop\arvii> & C:/Users/tloke/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/tloke/OneDrive/Desktop/arvii/10.py
30
PS C:\Users\tloke\OneDrive\Desktop\arvii> & C:/Users/tloke/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/tloke/OneDrive/Desktop/arvii/10.py
30
PS C:\Users\tloke\OneDrive\Desktop\arvii>
```

Task Description #2 –

Missing Error Handling

Task: Use AI to add proper error handling.

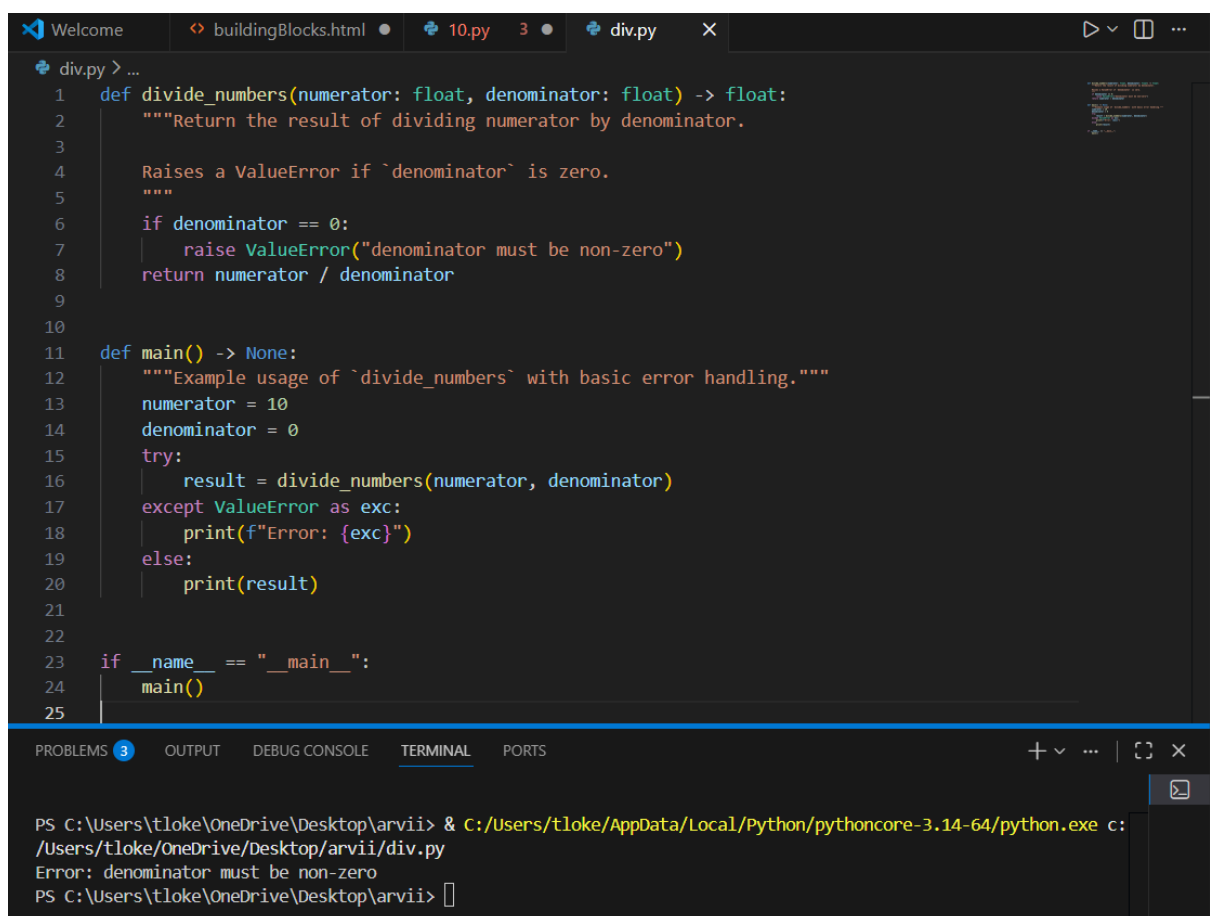
Sample Input Code:

```
def divide(a, b):  
  
    return a / b  
  
print(divide(10, 0))
```

Expected Output:

- Code with exception handling and clear error messages

CODE:



```
div.py > ...  
1 def divide_numbers(numerator: float, denominator: float) -> float:  
2     """Return the result of dividing numerator by denominator.  
3  
4     Raises a ValueError if `denominator` is zero.  
5     """  
6     if denominator == 0:  
7         raise ValueError("denominator must be non-zero")  
8     return numerator / denominator  
9  
10  
11 def main() -> None:  
12     """Example usage of `divide_numbers` with basic error handling."""  
13     numerator = 10  
14     denominator = 0  
15     try:  
16         result = divide_numbers(numerator, denominator)  
17     except ValueError as exc:  
18         print(f"Error: {exc}")  
19     else:  
20         print(result)  
21  
22  
23 if __name__ == "__main__":  
24     main()  
25
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\tloke\OneDrive\Desktop\arvii> & C:/Users/tloke/AppData/Local/Python/pythoncore-3.14-64/python.exe c:  
:/Users/tloke/OneDrive/Desktop/arvii/div.py  
Error: denominator must be non-zero  
PS C:\Users\tloke\OneDrive\Desktop\arvii>
```

Task Description #3:

Student Marks Processing System

The following program calculates total, average, and grade of a student, but it has poor readability, style issues, and no error handling.

```
marks=[78,85,90,66,88]
```

```
t=0
```

```
for i in marks:
```

```
    t=t+i
```

```
a=t/len(marks)
```

```
if a>=90:
```

```
    print("A")
```

```
elif a>=75:
```

```
    print("B")
```

```
elif a>=60:
```

```
    print("C")
```

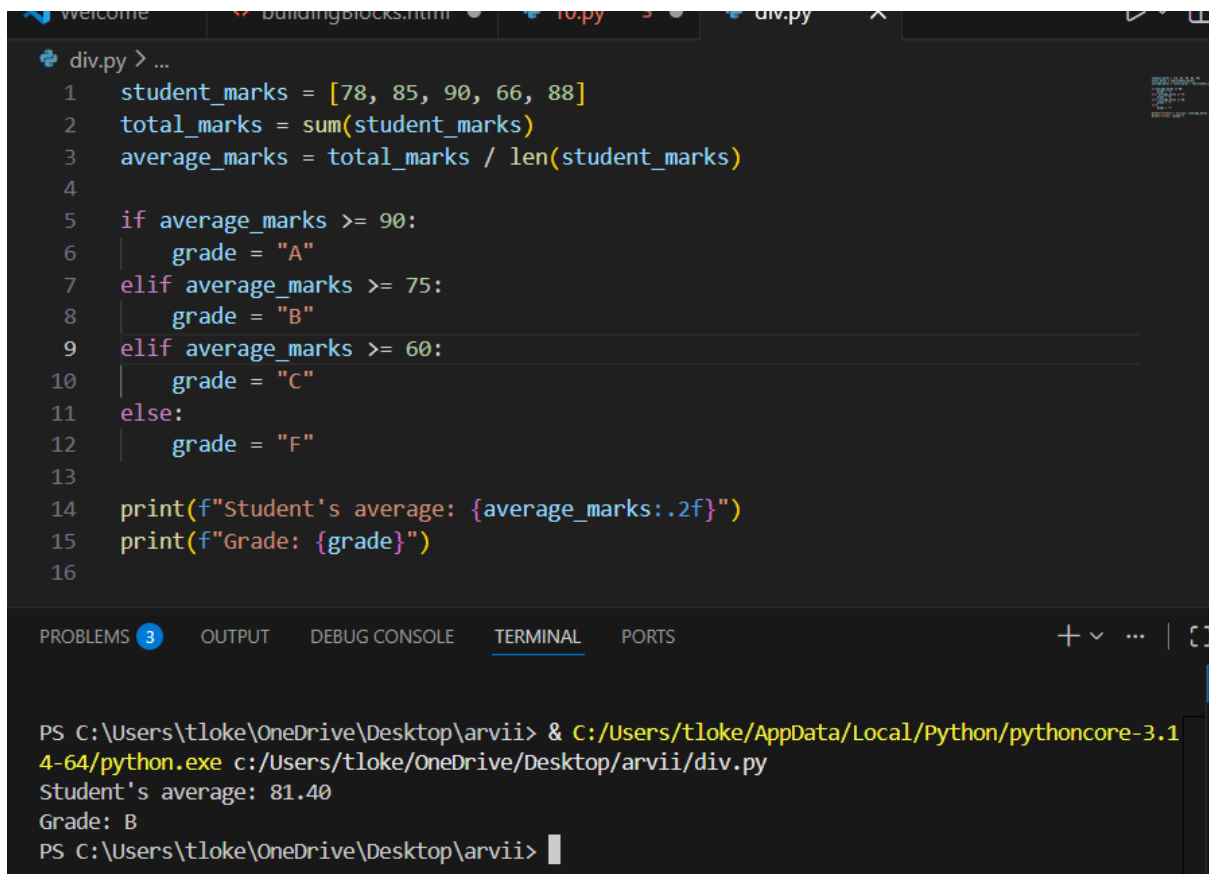
```
else:
```

```
    print("F")
```

Task:

- Use AI to refactor the code to follow PEP 8 standards.
- Add meaningful variable names, functions, and comments.
- Add basic input validation and documentation.

CODE:



The screenshot shows a Visual Studio Code editor window with a Python file named `div.py`. The code calculates the average of a list of student marks and assigns a grade based on the average. The terminal output shows the script being executed, displaying the average as 81.40 and the grade as B.

```
div.py > ...
1 student_marks = [78, 85, 90, 66, 88]
2 total_marks = sum(student_marks)
3 average_marks = total_marks / len(student_marks)
4
5 if average_marks >= 90:
6     grade = "A"
7 elif average_marks >= 75:
8     grade = "B"
9 elif average_marks >= 60:
10    grade = "C"
11 else:
12    grade = "F"
13
14 print(f"Student's average: {average_marks:.2f}")
15 print(f"Grade: {grade}")
16
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

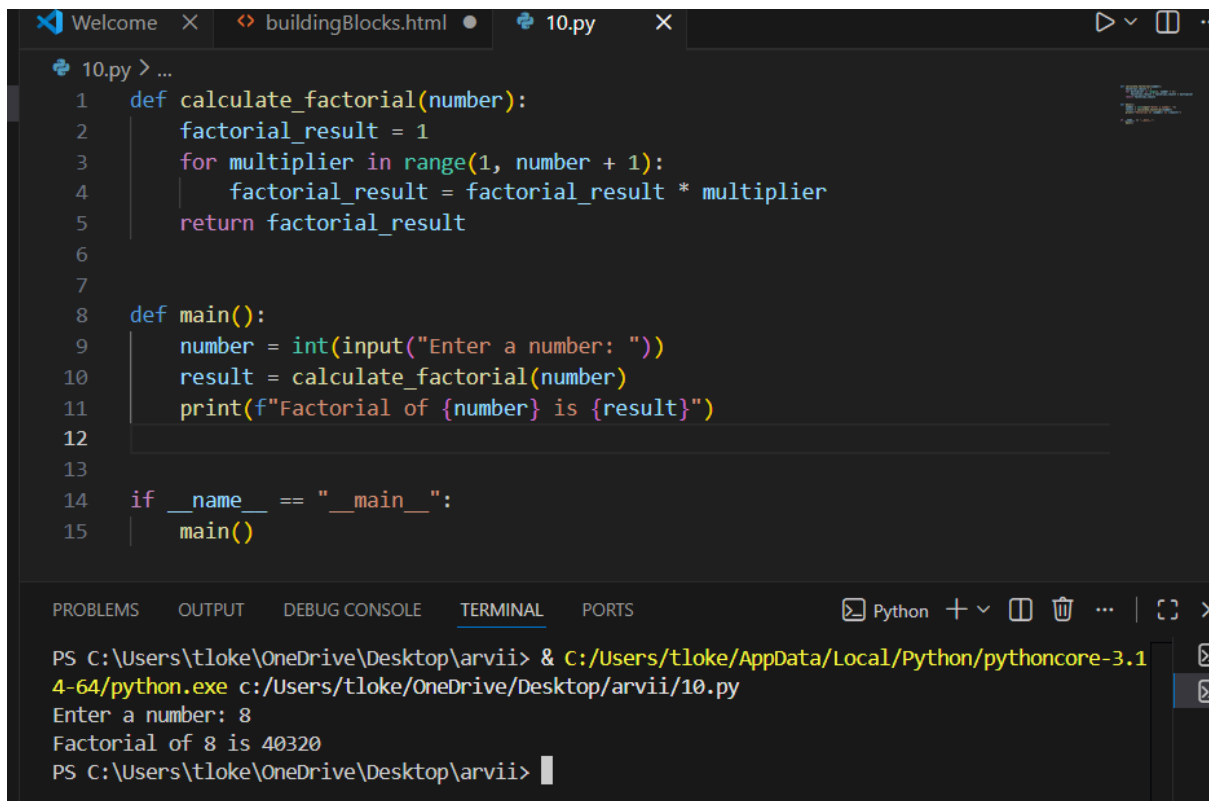
```
PS C:\Users\tloke\OneDrive\Desktop\arvii> & C:/Users/tloke/AppData/Local/Python/pythoncore-3.1
4-64/python.exe c:/Users/tloke/OneDrive/Desktop/arvii/div.py
Student's average: 81.40
Grade: B
PS C:\Users\tloke\OneDrive\Desktop\arvii>
```

Task Description #4:

Use AI to add docstrings and inline comments
to the following function.

```
def factorial(n):
    result = 1
    for i in range(1,n+1):
        result *= i
    return result
```

CODE:



```
10.py > ...
1  def calculate_factorial(number):
2      factorial_result = 1
3      for multiplier in range(1, number + 1):
4          factorial_result = factorial_result * multiplier
5      return factorial_result
6
7
8  def main():
9      number = int(input("Enter a number: "))
10     result = calculate_factorial(number)
11     print(f"Factorial of {number} is {result}")
12
13
14 if __name__ == "__main__":
15     main()
```

```
PS C:\Users\tloke\OneDrive\Desktop\arvii> & C:/Users/tloke/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/tloke/OneDrive/Desktop/arvii/10.py
Enter a number: 8
Factorial of 8 is 40320
PS C:\Users\tloke\OneDrive\Desktop\arvii>
```

Task Description #5:

Password Validation System (Enhanced)

The following Python program validates a password using only a minimum length check, which is insufficient for real-world security requirements.

```
pwd = input("Enter password: ")
```

```
if len(pwd) >= 8:
```

```
    print("Strong")
```

```
else:
```

```
    print("Weak")
```

Task:

1. Enhance password validation using AI assistance to include multiple security rules such as:

- o Minimum length requirement*
 - o Presence of at least one uppercase letter*
 - o Presence of at least one lowercase letter*
 - o Presence of at least one digit*
 - o Presence of at least one special character*
- 2. Refactor the program to:*
- o Use meaningful variable and function names*
 - o Follow PEP 8 coding standards*
 - o Include inline comments and a docstring*
- 3. Analyze the improvements by comparing the original and AI-enhanced versions in terms of:*
- o Code readability and structure*
 - o Maintainability and reusability*
 - o Security strength and robustness*
- 4. Justify the AI-generated changes, explaining why each added rule and refactoring decision improves the overall quality of the program*

CODE:

```
import re
```

```
def validate_password_strength(password):
```

```
    """
```

```
    Validates password strength based on multiple security criteria.
```

```
    Args:
```

```
        password (str): The password to validate
```

```
    Returns:
```

```
        tuple: (is_strong (bool), feedback (str)) - indicates if password is strong
```

and provides feedback on missing criteria

"""

Minimum password length requirement

minimum_length = 8

Check if password meets minimum length

if len(password) < minimum_length:

return False, f"Password must be at least {minimum_length} characters long"

Check for at least one uppercase letter

has_uppercase = bool(re.search(r"[A-Z]", password))

if not has_uppercase:

return False, "Password must contain at least one uppercase letter"

Check for at least one lowercase letter

has_lowercase = bool(re.search(r"[a-z]", password))

if not has_lowercase:

return False, "Password must contain at least one lowercase letter"

Check for at least one digit

has_digit = bool(re.search(r"\d", password))

if not has_digit:

return False, "Password must contain at least one digit"

Check for at least one special character

has_special_char = bool(re.search(r"[!@#\$%^&()_+!-=\[\]\{\};':\"\\|.,<>\/?]", password))*

if not has_special_char:

return False, "Password must contain at least one special character"

```

# All criteria met

return True, "Strong password - All security requirements met!"

def main():

    """Main function to run the password strength validator."""

    # Get password input from user

    user_password = input("Enter password: ")

    # Validate password strength

    is_strong, feedback = validate_password_strength(user_password)

    # Display result

    if is_strong:

        print(f"✓ {feedback}")

    else:

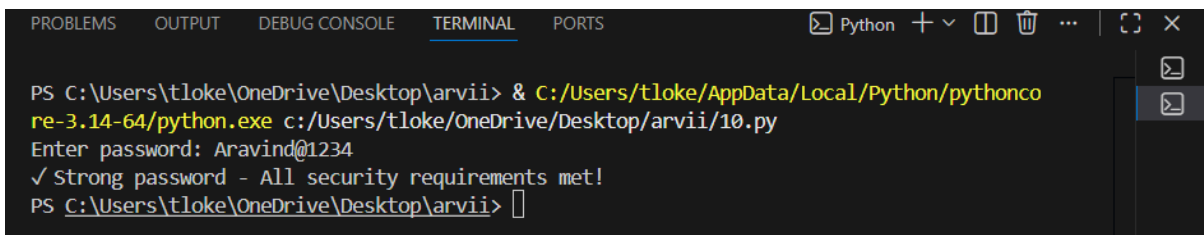
        print(f"X Weak password - {feedback}")

if __name__ == "__main__":

    main()

```

OUTPUT:



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Python + - [ ] [ ] ... [ ] [ ] [ ]

PS C:\Users\tloke\OneDrive\Desktop\arvii> & C:/Users/tloke/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/tloke/OneDrive/Desktop/arvii/10.py
Enter password: Aravind@1234
✓ Strong password - All security requirements met!
PS C:\Users\tloke\OneDrive\Desktop\arvii>

```